High-Performance C++ HTTP Server

Project name: PulseHTTP
Project type: Systems Programming / Backend Infrastructure / Network Engineering
Estimated development time: 5–7 weeks
Primary language: C++20
Target platform: Linux (Ubuntu 24.04 or equivalent)
Core technologies: C++20, POSIX sockets, epoll, HTTP/1.1, CMake, GoogleTest, Google Benchmark, Docker, Prometheus
Supporting technologies: AddressSanitizer, UndefinedBehaviorSanitizer, ThreadSanitizer, perf, wrk, hey, GitHub Actions
Target roles: Backend Software Engineer, Systems Engineer, Infrastructure Engineer, C++ Software Engineer, Performance Engineer

---

1. Executive Summary

1.1 Project Overview

PulseHTTP is a high-performance, multithreaded HTTP/1.1 server built from scratch in modern C++.

The server will use nonblocking TCP sockets, Linux's `epoll` event notification mechanism, and a bounded worker pool to handle many simultaneous connections without allocating one thread per client.

The system will support persistent HTTP connections, incremental request parsing, asynchronous network I/O, configurable resource limits, graceful shutdown, and production-style observability.

The project will also include a benchmarking framework for evaluating request throughput, tail latency, CPU utilization, memory consumption, and behavior under high concurrency.

Rather than simply demonstrating that a server can respond to HTTP requests, PulseHTTP will investigate the engineering tradeoffs between different concurrency architectures.

1.2 Problem Statement

A basic HTTP server can accept a TCP connection, read a request, generate a response, and close the socket.

However, this design becomes inefficient when thousands of clients connect simultaneously.

A traditional thread-per-connection server may encounter:

- High memory consumption from thread stacks.
- Excessive context switching.
- Poor performance under slow clients.
- Resource exhaustion when connections increase.
- Difficulties implementing backpressure.
- Increased tail latency under CPU saturation.

An event-driven server can handle large numbers of connections with relatively few threads, but introduces new engineering challenges.

These include partial reads and writes, connection state management, race conditions, request pipelining, socket lifecycle management, and safe coordination between event loops and worker threads.

PulseHTTP will address these challenges through a carefully designed reactor architecture and measurable performance experiments.

1.3 Primary Objective

Build a reliable, event-driven HTTP/1.1 server that efficiently manages concurrent TCP connections and demonstrates measurable performance characteristics under controlled workloads.

The server should answer questions such as:

- How does an `epoll`-based architecture compare with a thread-per-connection implementation?
- How many simultaneous connections can the server maintain within a fixed memory budget?
- How does worker pool size affect throughput and p95/p99 latency?
- What happens when clients send incomplete or fragmented HTTP requests?
- How should the server respond when its worker queue becomes saturated?
- How does HTTP keep-alive influence connection overhead?
- How can partial writes be handled without blocking the event loop?
- What is the performance cost of excessive memory allocation?
- How does the server behave during shutdown while requests are still active?

2. Scope and Non-Goals

2.1 In Scope

The project will implement:

1. Nonblocking TCP sockets.
2. Linux `epoll`-based event notification.
3. HTTP/1.1 request parsing.
4. HTTP keep-alive connections.
5. Incremental request buffering.
6. Partial read and write handling.
7. A bounded worker thread pool.
8. Request routing and handlers.
9. Static file serving.
10. Connection and request timeouts.
11. Resource limits and backpressure.
12. Graceful shutdown.
13. Structured logging and operational metrics.
14. Automated unit and integration tests.
15. Load testing and performance profiling.
16. Reproducible benchmark reports.

2.2 Out of Scope

The initial version will not implement:

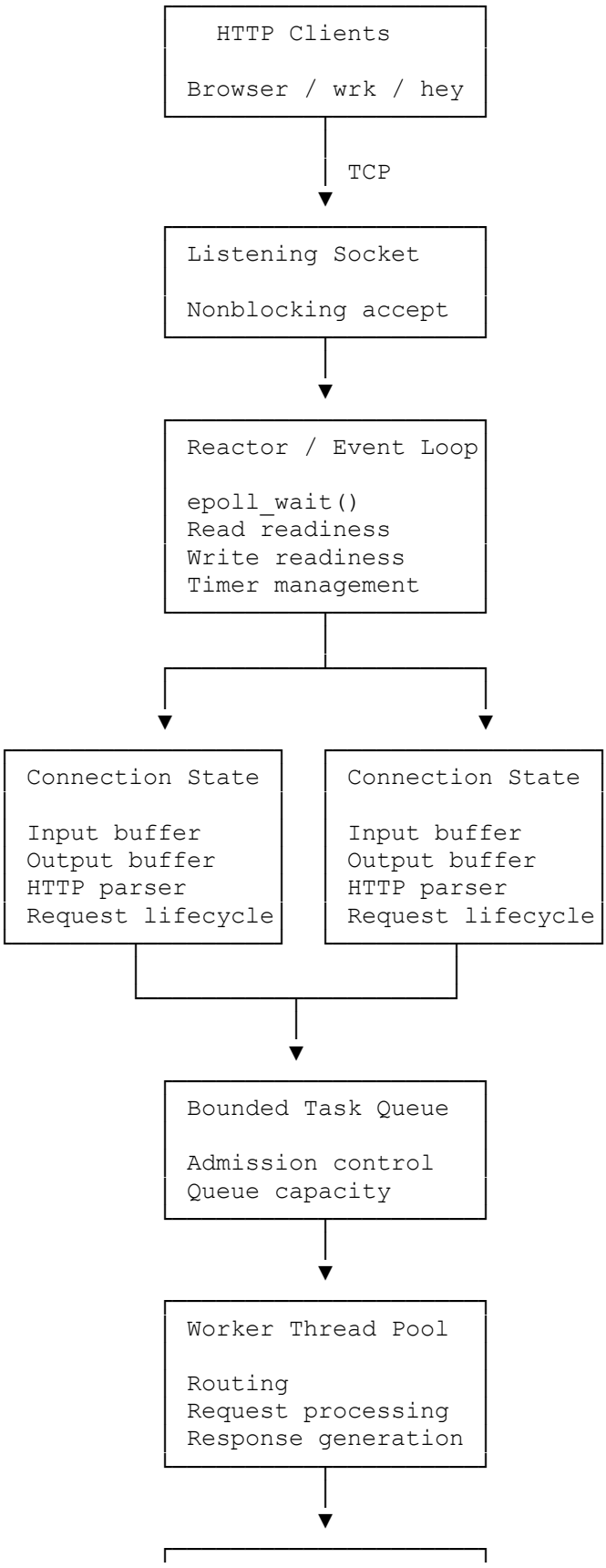
- HTTP/2 or HTTP/3.
- TLS encryption directly inside the server.
- WebSocket support.
- Reverse proxy functionality.
- Distributed load balancing.
- Dynamic scripting runtimes.
- Full CGI/FastCGI support.
- A production-grade general-purpose HTTP framework.
- Custom kernel networking implementations.
- `io_uring` as the primary I/O mechanism.

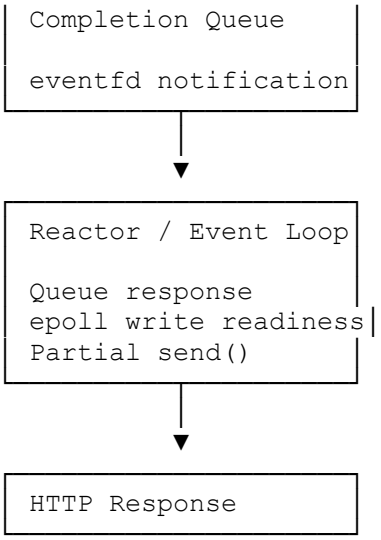
TLS may be added later through a reverse proxy or an optional OpenSSL integration.

The project is intended as a systems-engineering portfolio project, not as a replacement for Nginx.

3. System Architecture

3.1 High-Level Architecture


HTTP Clients
Browser / wrk / hey
TCP
Listening Socket
Nonblocking accept
Reactor / Event Loop
epoll_wait()
Read readiness
Write readiness
Timer management
Connection State
Input buffer
Output buffer
HTTP parser
Request lifecycle
Connection State
Input buffer
Output buffer
HTTP parser
Request lifecycle
Bounded Task Queue
Admission control
Queue capacity
Worker Thread Pool
Routing
Request processing
Response generation



3.2 Architectural Model

PulseHTTP will use a **Reactor + Worker Pool** design.

The reactor owns network I/O and connection state.

Workers execute application-level request handlers.

This separation ensures that expensive application work does not block socket readiness processing.

Reactor Responsibilities

- Accept incoming connections.
- Register sockets with `epoll`.
- Read available socket data.
- Feed data into incremental HTTP parsers.
- Apply input/output buffer limits.
- Dispatch complete requests to workers.
- Receive completed responses.
- Write response bytes without blocking.
- Close idle or invalid connections.
- Manage socket lifetime.

Worker Responsibilities

- Receive parsed requests.
- Match routes.
- Execute request handlers.
- Generate HTTP responses.

- Return completed results to the reactor.

Workers should not directly modify reactor-owned connection state.

Communication Between Reactor and Workers

Use:

- A bounded, thread-safe request queue.
- A thread-safe completion queue.
- An `eventfd` to wake the reactor when completed responses are available.

The reactor will associate each dispatched request with a connection identifier and generation number.

When a worker finishes, the reactor checks whether the corresponding connection is still valid before attaching the response.

This prevents stale worker completions from writing to a closed or reused socket.

4. Technology Stack

Component	Technology	Purpose
Programming language	C++20	Modern systems programming
Operating system	Linux	Native networking APIs
TCP networking	POSIX sockets	Socket creation and I/O
Event notification	epoll	Read/write readiness
Reactor wake-up	eventfd	Worker completion notifications
Timeout management	timerfd or monotonic timer heap	Connection deadlines
Concurrency	std::thread / std::jthread	Worker execution
Synchronization	mutex, condition_variable	Bounded queues
Build system	CMake	Compilation and dependency management
Unit testing	GoogleTest	Parser and component tests
Benchmarking	wrk, hey, Google Benchmark	Load and microbenchmarks
Profiling	perf	CPU profiling
Memory analysis	ASan, UBSan, Valgrind	Memory and undefined behavior detection

Race analysis	ThreadSanitizer	Concurrency validation
Metrics	Prometheus exposition format	Operational monitoring
Containers	Docker	Reproducible execution
CI	GitHub Actions	Build and test automation

5. TCP Networking Layer

5.1 Listening Socket

The server will create a TCP listening socket using Linux networking APIs.

Initialization sequence:

1. Create a socket with `socket()`.
2. Enable `SO_REUSEADDR`.
3. Set the socket to nonblocking mode.
4. Bind to the configured IP address and port.
5. Call `listen()`.
6. Register the listening socket with `epoll`.

Prefer atomic nonblocking and close-on-exec flags when supported.

5.2 Accepting Connections

When the listening socket becomes readable, the reactor should accept pending connections until the kernel reports `EAGAIN` or `EWOULDBLOCK`.

Each accepted socket will be:

- Nonblocking.
- Registered with `epoll`.
- Assigned a unique connection identity.
- Associated with input/output buffers.
- Subject to configured connection limits.
- Tracked for idle timeout enforcement.

5.3 Partial Reads

TCP provides a byte stream, not a sequence of complete HTTP messages.

A single HTTP request may arrive in multiple reads.

Likewise, one read may contain multiple requests.

The server must support both cases.

Example:

```
Read 1:
GET /api/health HT

Read 2:
TP/1.1\r\nHost: localhost\r\n

Read 3:
\r\n
```

The parser must preserve incomplete input until the full request is available.

5.4 Partial Writes

A call to `send()` may write fewer bytes than requested.

PulseHTTP must track how much of the response has already been transmitted.

Example:

```
Response size: 16 KB

send() #1 -> 4 KB
send() #2 -> EAGAIN
EPOLLOUT event
send() #3 -> 8 KB
send() #4 -> 4 KB

Response complete
```

The event loop must never assume a single `send()` transmits the entire response.

5.5 Edge-Triggered vs Level-Triggered epoll

Start with **level-triggered epoll** to establish correct behavior.

After the baseline server is stable, add an edge-triggered configuration for comparison.

Edge-triggered mode requires draining readable and writable sockets until `EAGAIN` to avoid missing progress opportunities.

Benchmark both modes rather than assuming one is always faster.

6. Connection Lifecycle Management

6.1 Connection State

Each connection will maintain:

```
struct Connection {
    ConnectionId id;
    int fd;

    std::vector<char> input_buffer;
    std::vector<char> output_buffer;

    size_t write_offset = 0;

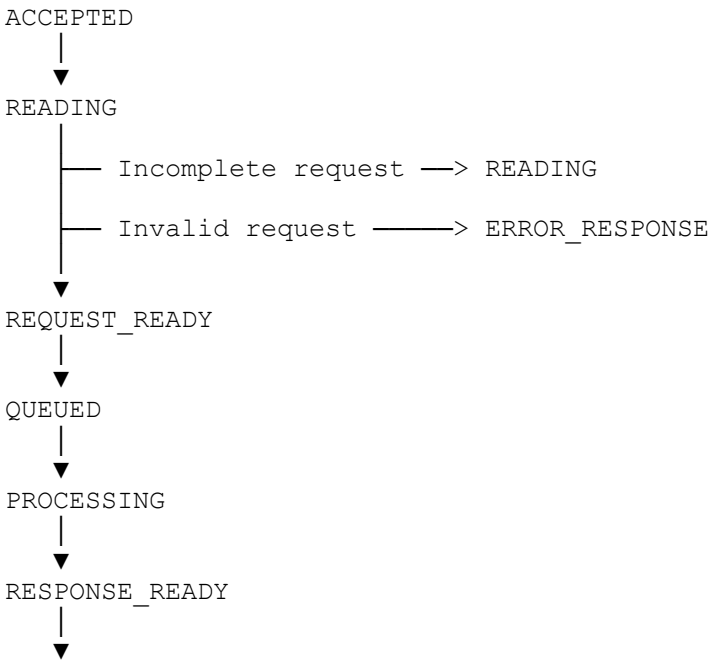
    HttpParser parser;

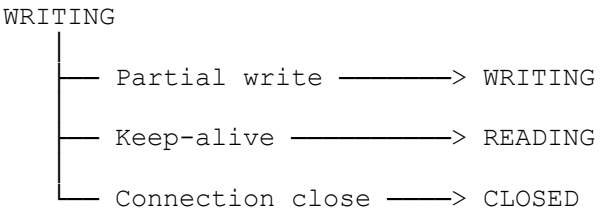
    ConnectionState state;

    std::chrono::steady_clock::time_point last_activity;
};
```

The final implementation should use RAII for socket ownership and should avoid copying large buffers unnecessarily.

6.2 State Machine





Additional transitions will handle timeouts, client disconnects, shutdown, and errors.

6.3 Connection Identity

Do not identify an asynchronous request solely by its file descriptor.

Linux may reuse file descriptor numbers after a socket closes.

Instead, assign a monotonically increasing connection identifier or generation token.

Worker completions must be checked against the current connection identity before use.

6.4 Resource Limits

Provide configuration for:

- Maximum active connections.
- Maximum request-header size.
- Maximum request-body size.
- Maximum input-buffer size.
- Maximum output-buffer size.
- Maximum worker queue depth.
- Idle connection timeout.
- Header read timeout.
- Request processing timeout.
- Maximum requests per keep-alive connection.

Resource limits should be enforced explicitly and tested.

7. HTTP/1.1 Protocol Implementation

7.1 Supported Request Methods

Initial methods:

- GET
- HEAD
- POST

Other methods should receive an appropriate response based on whether they are recognized and supported.

7.2 HTTP Request Parser

Implement an incremental parser capable of handling:

1. Request line.
2. HTTP version.
3. Headers.
4. Message framing.
5. Request body.
6. Multiple requests on one connection.

The parser should be stateful and should not assume that an entire request arrives in one read.

7.3 Required Protocol Behavior

Support:

- HTTP/1.1 persistent connections.
- `Content-Length` request bodies.
- `Connection: close`.
- Multiple sequential requests over one socket.
- Proper response status lines.
- Valid `Content-Length` response headers.
- HEAD responses without a message body.
- HTTP pipelining with ordered responses, subject to documented limits.

A conservative first implementation may permit only one active handler per connection while buffering subsequent requests. This simplifies ordering and backpressure.

7.4 Request Framing and Security

HTTP request parsing introduces security-sensitive edge cases.

The implementation must explicitly handle:

- Conflicting or duplicated `Content-Length`.
- Unsupported transfer codings.
- Requests containing both `Transfer-Encoding` and `Content-Length`.

- Malformed header names or values.
- Invalid request-line syntax.
- Embedded NUL bytes.
- Excessively large headers.
- Incomplete request bodies.
- Unexpected extra bytes.
- Ambiguous message boundaries.

For the MVP, reject unsupported chunked request bodies rather than attempting to partially support them.

Do not forward ambiguous requests to another HTTP server.

This reduces request-smuggling risks associated with inconsistent HTTP framing.

7.5 HTTP Response Builder

Create a reusable response builder supporting:

- Status code.
- Reason phrase.
- Headers.
- Body.
- Content type.
- Content length.
- Connection behavior.

Example:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 15
Connection: keep-alive

{"status":"ok"}
```

The response builder should calculate the correct byte length and use CRLF line endings.

8. Request Routing and Application Layer

8.1 Router

Implement a lightweight router that maps method/path combinations to handlers.

Example routes:

Endpoint	Method	Purpose
`/health`	GET	Health check
`/hello`	GET	Plain-text response
`/api/echo`	POST	Echo request body
`/api/compute`	GET	CPU-bound benchmark handler
`/metrics`	GET	Prometheus metrics
`/static/*`	GET	Static file serving

8.2 Handler Interface

A possible interface:

```
using Handler = std::function<HttpResponse(const HttpRequest&)>;

class Router {
public:
    void add_route(
        HttpMethod method,
        std::string path,
        Handler handler
    );

    HttpResponse dispatch(const HttpRequest& request) const;
};
```

The router should support immutable route registration before server startup, avoiding unnecessary synchronization during request dispatch.

8.3 Static File Serving

Support files under a configured document root.

Requirements:

- Reject directory traversal.
- Prevent symlink escapes.
- Set correct content length.
- Support common MIME types.
- Handle missing files.
- Enforce file-size limits.
- Avoid loading arbitrarily large files into memory.

Use a secure root-relative file-opening strategy. A simple string-prefix path check is insufficient.

For an advanced optimization, compare buffered file reads with Linux `sendfile()` for static responses.

9. Worker Thread Pool

9.1 Motivation

The event loop must remain responsive while application handlers perform work.

A bounded worker pool provides predictable concurrency and avoids creating unlimited threads.

9.2 Worker Pool Design

```
class ThreadPool {
public:
    explicit ThreadPool(
        size_t thread_count,
        size_t queue_capacity
    );

    bool submit(Task task);

    void shutdown();

private:
    std::vector<std::jthread> workers_;

    std::mutex mutex_;
    std::condition_variable cv_;

    std::deque<Task> tasks_;

    size_t capacity_;
    bool stopping_ = false;
};
```

The queue must reject new tasks when capacity is exhausted.

Workers should use a clear shutdown protocol and complete or cancel pending tasks according to the configured policy.

9.3 Backpressure

When the worker queue is full, the server must not block the reactor waiting for capacity.

Possible strategies:

- Return HTTP 503 Service Unavailable.
- Temporarily suspend reads from overloaded connections.
- Apply connection-level admission limits.
- Close connections that exceed resource limits.

The MVP should implement a documented, deterministic overload policy.

Measure overload outcomes separately from normal request latency.

9.4 Thread Pool Experiments

Compare worker counts:

- 1
- 2
- 4
- 8
- 16

Use both lightweight handlers and CPU-bound handlers.

Measure:

- Requests/sec.
- p95/p99 latency.
- Queue depth.
- CPU utilization.
- Context switches.
- Error and rejection rates.

The optimal worker count will depend on the workload and available CPU cores.

10. Memory Management and Performance Optimization

10.1 RAII Resource Ownership

Use RAII wrappers for:

- Socket file descriptors.
- epoll descriptors.

- eventfd descriptors.
- timerfd descriptors.
- File handles.

This prevents resource leaks during exceptions and early returns.

10.2 Buffer Management

Start with `std::vector<char>` or `std::string` buffers.

Avoid premature custom allocator development.

Potential optimizations:

- Reuse allocated buffer capacity.
- Minimize unnecessary copying.
- Avoid repeatedly erasing from the front of large buffers.
- Use offsets or ring buffers for consumed data.
- Reserve small, predictable buffer capacities.
- Track peak buffered bytes per connection.

10.3 Static File Optimization

Compare:

Buffered I/O

Read file contents into userspace buffers and send them over the socket.

sendfile()

Transfer file data using the Linux kernel's file-to-socket path.

Measure performance rather than assuming zero-copy behavior always produces a significant benefit.

10.4 Profiling

Use Linux `perf` to investigate:

- CPU hotspots.
- Context switching.
- Allocation overhead.
- System call frequency.
- Lock contention.

- Cache misses where available.

Optimization decisions should be driven by profiling evidence.

11. Observability

11.1 Metrics

Expose Prometheus-compatible metrics.

Suggested metrics:

Metric	Type
`http_requests_total`	Counter
`http_request_duration_seconds`	Histogram
`http_active_connections`	Gauge
`http_connections_accepted_total`	Counter
`http_connections_closed_total`	Counter
`http_worker_queue_depth`	Gauge
`http_worker_queue_rejections_total`	Counter
`http_bytes_received_total`	Counter
`http_bytes_sent_total`	Counter
`http_parse_errors_total`	Counter
`http_connection_timeouts_total`	Counter
`http_response_write_errors_total`	Counter

Avoid high-cardinality labels such as individual connection IDs or raw arbitrary URLs.

11.2 Logging

Support structured logs with:

- Timestamp.
- Log level.
- Connection ID.
- Request ID.
- HTTP method.
- Route.
- Status code.

- Duration.
- Error category.

Logging must not block the reactor during normal high-throughput operation.

11.3 Health Endpoints

Implement:

- `/health` for process liveness.
- `/ready` for readiness to accept traffic.
- `/metrics` for operational metrics.

Readiness should become false during graceful shutdown.

12. Testing Strategy

12.1 Unit Tests

Test the HTTP parser with:

- Complete requests.
- Fragmented request lines.
- Fragmented headers.
- Fragmented bodies.
- Multiple requests in one buffer.
- Invalid HTTP versions.
- Oversized headers.
- Conflicting framing headers.
- Malformed content lengths.
- Keep-alive semantics.
- HEAD request behavior.

Test the worker pool with:

- Empty queue.
- Full queue.
- Concurrent producers.
- Concurrent consumers.
- Shutdown with pending tasks.
- Exception handling.

12.2 Integration Tests

Launch the actual server process and test:

- GET and POST requests.
- Keep-alive connections.
- Multiple requests over one socket.
- HTTP pipelining.
- Partial client writes.
- Slow clients.
- Abrupt disconnects.
- Large valid bodies.
- Invalid request framing.
- Idle timeouts.
- Graceful shutdown.
- Worker saturation.

12.3 Stress Tests

Generate high concurrency and repeated connection churn.

Verify:

- No crashes.
- No use-after-free.
- No double close.
- No stale response delivery.
- No unbounded queue growth.
- No unbounded memory growth.
- No file descriptor leaks.
- No corrupted HTTP responses.

12.4 Sanitizer Testing

Run appropriate builds with:

- AddressSanitizer.
- UndefinedBehaviorSanitizer.
- ThreadSanitizer.

ThreadSanitizer should generally run in a separate build from AddressSanitizer.

Not all sanitizer configurations are supported equally across operating systems and compilers.

12.5 Fuzz Testing

Use a coverage-guided fuzzing harness for the HTTP parser.

Fuzz targets should include:

- Request-line parsing.
- Header parsing.
- Message framing.
- Incremental input handling.
- Malformed and truncated requests.

The parser should reject invalid inputs safely without crashes, out-of-bounds accesses, or uncontrolled allocations.

13. Benchmarking Methodology

13.1 Benchmark Objectives

Measure both performance and reliability.

A high requests-per-second result is not useful if requests are dropped, responses are corrupted, or latency becomes unacceptable.

13.2 Benchmark Environments

Record:

- CPU model.
- Physical/logical core count.
- RAM.
- Linux kernel version.
- Compiler and optimization flags.
- Server commit SHA.
- Network topology.
- File descriptor limits.
- TCP backlog settings.
- Benchmark tool and version.
- Server configuration.
- Client concurrency.
- Request duration.
- Warm-up duration.

13.3 Workload Types

Workload A — Small Static Response

```
GET /hello
```

Purpose: Measure baseline HTTP processing overhead.

Workload B — JSON Response

```
GET /health
```

Purpose: Measure routing and response serialization overhead.

Workload C — CPU-Bound Handler

```
GET /api/compute
```

Purpose: Evaluate worker pool scaling and queue saturation.

Workload D — Static File Transfer

```
GET /static/sample.bin
```

Purpose: Evaluate network I/O, buffering, and file transfer performance.

Use several file sizes, such as 4 KB, 64 KB, and 1 MB.

Workload E — Keep-Alive

Compare persistent connections with new connections per request.

Purpose: Measure connection establishment overhead and connection management efficiency.

Workload F — Slow Clients

Simulate clients that read responses slowly or send requests incrementally.

Purpose: Verify nonblocking behavior and bounded resource consumption.

13.4 Primary Metrics

- Successful requests/sec.
- p50 response latency.
- p95 response latency.

- p99 response latency.
- Error rate.
- CPU utilization.
- Resident memory usage.
- Active connection count.
- Context switches/sec.
- File descriptor usage.
- Worker queue depth.

13.5 Benchmark Controls

For each important comparison:

1. Use the same hardware.
2. Use the same client/server topology.
3. Use the same workload and request distribution.
4. Warm up the server.
5. Run multiple repetitions.
6. Preserve raw results.
7. Report median and observed variation.
8. Include errors and timeouts.
9. Avoid running unrelated heavy workloads.
10. Separate local-loopback results from networked results.

A single impressive throughput number is not sufficient evidence of performance superiority.

14. Performance Experiments

Experiment 1 — Thread-per-Connection vs epoll

Question

Does an event-driven architecture use fewer resources under high connection concurrency?

Baseline

Implement a deliberately simple but correct thread-per-connection server with bounded admission and equivalent HTTP response behavior.

Candidate

PulseHTTP using nonblocking sockets and `epoll`.

Compare

- Throughput.
- Tail latency.
- CPU utilization.
- Context switches.
- Memory consumption.
- Maximum sustainable connections.
- Error rate.

Deliverable

A report explaining where the event-driven architecture helps and where its overhead may outweigh benefits.

Experiment 2 — Worker Pool Scaling

Question

How does worker count affect CPU-bound request processing?

Variables

Worker threads: 1, 2, 4, 8, 16.

Deliverable

Throughput and tail-latency curves with queue-depth analysis.

Experiment 3 — Keep-Alive Efficiency

Question

How much overhead is saved by reusing TCP connections?

Compare

- One request per connection.
- Multiple requests per connection.

Deliverable

Throughput, latency, connection-count and CPU-cost comparison.

Experiment 4 — Static File Transfer

Question

How does buffered I/O compare with `sendfile()`?

Compare

- Buffered reads and writes.
- `sendfile()`.

Deliverable

Performance comparison across file sizes.

Experiment 5 — Backpressure Under Overload

Question

What happens when incoming work exceeds server capacity?

Configuration

Increase client request rate beyond the server's sustainable processing rate.

Measure

- Worker queue depth.
- Rejected requests.
- Successful throughput.
- Tail latency.
- Memory usage.
- Recovery time after load decreases.

Deliverable

A report showing whether the server degrades predictably instead of exhausting resources.

---

15. Suggested Repository Structure

```
pulsehttp/
├── README.md
├── CMakeLists.txt
├── Dockerfile
├── .clang-format
├── .clang-tidy
├── .github/
│   └── workflows/
│       └── ci.yml
├── include/
│   └── pulsehttp/
│       ├── server.h
│       ├── reactor.h
│       ├── connection.h
│       ├── socket.h
│       ├── epoll_wrapper.h
│       ├── http_parser.h
│       ├── http_request.h
│       ├── http_response.h
│       ├── router.h
│       ├── thread_pool.h
│       ├── bounded_queue.h
│       ├── metrics.h
│       └── config.h
├── src/
│   ├── main.cpp
│   ├── server.cpp
│   ├── reactor.cpp
│   ├── connection.cpp
│   ├── socket.cpp
│   ├── epoll_wrapper.cpp
│   ├── http_parser.cpp
│   ├── http_response.cpp
│   ├── router.cpp
│   ├── thread_pool.cpp
│   ├── metrics.cpp
│   └── config.cpp
├── tests/
│   ├── test_http_parser.cpp
│   ├── test_http_framing.cpp
│   ├── test_thread_pool.cpp
│   ├── test_connection.cpp
│   ├── test_keep_alive.cpp
│   ├── test_backpressure.cpp
│   ├── test_shutdown.cpp
│   └── integration/
├── benchmarks/
│   ├── wrk/
```

```
│   ├── hey/
│   ├── scripts/
│   ├── baselines/
│   └── reports/
├── fuzz/
│   └── http_parser_fuzz.cpp
├── monitoring/
│   ├── prometheus.yml
│   └── grafana/
├── docs/
│   ├── architecture.md
│   ├── protocol.md
│   ├── concurrency.md
│   ├── benchmarks.md
│   ├── performance-analysis.md
│   └── limitations.md
└── examples/
    ├── basic_server.cpp
    └── custom_routes.cpp
```

---

16. Development Roadmap

Week 1 — TCP Server and Event Loop

Goal: Build a functioning nonblocking TCP server.

Tasks:

- Initialize CMake project.
- Implement RAII file descriptor wrapper.
- Create nonblocking listening socket.
- Implement epoll wrapper.
- Accept concurrent TCP connections.
- Read incoming bytes.
- Return a fixed HTTP response.
- Implement basic connection cleanup.
- Add unit tests and CI.

Deliverable: A minimal event-driven HTTP server.

Acceptance criteria:

- Handles multiple simultaneous connections.
- Does not block on socket reads or writes.
- Handles partial writes.
- Closes connections without leaking descriptors.

Week 2 — HTTP Parser and Keep-Alive

Goal: Implement reliable HTTP/1.1 request processing.

Tasks:

- Build incremental request parser.
- Implement HTTP request/response types.
- Support GET, HEAD and POST.
- Support Content-Length bodies.
- Implement keep-alive.
- Implement request pipelining.
- Add protocol error handling.
- Add parser unit tests.
- Add HTTP framing security tests.

Deliverable: Functional HTTP/1.1 server with persistent connections.

Acceptance criteria:

- Fragmented requests parse correctly.
- Multiple requests per connection work.
- Invalid framing is rejected.
- Response order is preserved.
- No invalid request causes a crash.

Week 3 — Worker Pool and Backpressure

Goal: Separate network I/O from application processing.

Tasks:

- Implement bounded worker pool.
- Implement task submission.
- Add completion queue.
- Add eventfd notification.
- Introduce connection generation IDs.
- Add router and request handlers.
- Implement overload policy.
- Add worker queue metrics.
- Test disconnects during processing.

Deliverable: Multithreaded reactor-based HTTP server.

Acceptance criteria:

- Event loop remains responsive during CPU-bound work.
- Queue capacity is enforced.
- Stale completions cannot write to reused sockets.
- Worker shutdown is deterministic.

Week 4 — Reliability and Production Features

Goal: Make the server robust under adverse conditions.

Tasks:

- Implement idle/header/body timeouts.
- Implement graceful shutdown.
- Add request and connection limits.
- Add static file serving.
- Add Prometheus metrics.
- Add structured logging.
- Add sanitizer builds.
- Add parser fuzzing.
- Add stress and slow-client tests.

Deliverable: Reliable server with operational visibility.

Acceptance criteria:

- Resource limits remain bounded.
- Slow clients cannot indefinitely consume resources.
- No known memory-safety failures under tests.
- Graceful shutdown handles active connections.
- Metrics reflect actual server behavior.

Week 5 — Benchmarking and Profiling

Goal: Establish trustworthy performance baselines.

Tasks:

- Implement or configure thread-per-connection baseline.
- Run wrk/hey benchmarks.
- Test keep-alive.
- Test concurrency scaling.
- Test worker counts.
- Capture CPU and memory metrics.
- Use perf to identify hotspots.
- Publish raw benchmark artifacts.

Deliverable: Reproducible baseline performance report.

Acceptance criteria:

- Multiple benchmark repetitions completed.
- Workload definitions are preserved.
- Error counts are included.
- Results are tied to hardware and configuration metadata.

Week 6 — Optimization

Goal: Demonstrate evidence-based improvements.

Tasks:

- Optimize the largest measured bottleneck.
- Evaluate buffer reuse.
- Compare buffered file I/O with sendfile().
- Compare epoll LT and ET modes.
- Rerun benchmarks.
- Explain performance changes.
- Check for reliability regressions.

Deliverable: Before/after optimization case study.

Acceptance criteria:

- Changes are isolated and reproducible.
- Improvements are supported by measurements.
- No significant correctness regression is introduced.
- Tradeoffs are documented.

Week 7 — Portfolio Polish

Goal: Prepare the project for technical review and interviews.

Tasks:

- Finalize README.
- Document architecture and design decisions.
- Publish benchmark charts.
- Include test and sanitizer results.
- Provide Docker instructions.
- Prepare a short demo.
- Write resume bullets.

- Prepare interview explanations.

Deliverable: Portfolio-ready C++ systems project.

---

17. Minimum Viable Product

The MVP should contain:

1. Nonblocking TCP sockets.
2. An epoll-based event loop.
3. An incremental HTTP/1.1 parser.
4. GET and POST support.
5. Persistent HTTP connections.
6. Partial read/write handling.
7. A bounded worker thread pool.
8. Connection-safe asynchronous completions.
9. A basic router.
10. Resource limits.
11. Unit and integration tests.
12. One reproducible load-test report.

Do not begin with a custom memory allocator, HTTP/2, TLS, or a complicated framework API.

These features would increase complexity without proving the project's core engineering objective.

---

18. Definition of Done

The project is considered complete when the following are demonstrated.

Category	Requirement
Networking	Correct nonblocking TCP I/O
HTTP	Valid HTTP/1.1 parsing and responses
Concurrency	Event-driven I/O with bounded workers
Reliability	Safe socket lifecycle and graceful shutdown
Security	Strict message framing and resource limits
Testing	Parser, concurrency and integration tests
Memory safety	Sanitizer and fuzz-testing evidence

Performance	Reproducible throughput and tail-latency benchmarks
Optimization	At least one measured tradeoff or improvement
Documentation	Architecture, methodology and limitations published

No arbitrary throughput target is required.

A valid benchmark showing a regression or no improvement can still demonstrate strong engineering judgment if the methodology and analysis are sound.

---

19. Expected Engineering Skills

Completing PulseHTTP will demonstrate practical understanding of:

- TCP socket programming.
- Linux kernel I/O interfaces.
- Event-driven server architecture.
- Reactor design patterns.
- Multithreaded concurrency.
- Synchronization and bounded queues.
- HTTP protocol implementation.
- Incremental parsing.
- Memory ownership and RAII.
- Socket lifetime and stale-completion prevention.
- Backpressure and overload control.
- Tail-latency analysis.
- CPU profiling.
- Performance benchmarking.
- Systems-level testing and debugging.

These skills transfer directly to backend infrastructure, networking, databases, storage engines, and high-performance C++ systems.

---

20. Potential Resume Experience

PulseHTTP — High-Performance C++ HTTP Server
C++20, Linux, epoll, POSIX Sockets, CMake, GoogleTest, Docker

Suggested bullets after implementation:

- Engineered a multithreaded HTTP/1.1 server in C++20 using nonblocking sockets, Linux epoll and a reactor-based architecture to support concurrent persistent connections.
- Implemented incremental HTTP parsing, bounded worker scheduling, connection lifecycle management and backpressure to ensure reliable operation under fragmented requests and high concurrency.
- Profiled and benchmarked server throughput, p95/p99 latency and resource utilization across concurrency and I/O configurations, validating correctness with integration tests and memory sanitizers.

Once real benchmark results are available, replace the final bullet with a concrete, verified performance result.

---

21. Relationship to Existing Portfolio

Distributed Task Queue

Focus: Reliable distributed workload execution.

Core concepts:

- Durable jobs.
- Worker leases.
- Idempotency.
- Failure recovery.
- Transactional persistence.

EvalScope

Focus: LLM quality measurement and evaluation infrastructure.

Core concepts:

- Experiment orchestration.
- Model quality.
- Observability.
- Regression detection.
- LLM-as-judge.

LLM Inference Performance Lab

Focus: Model-serving performance measurement.

Core concepts:

- Inference throughput.
- Streaming latency.
- GPU telemetry.
- Batching.
- Resource efficiency.

PulseHTTP

Focus: Low-level networking and server performance.

Core concepts:

- Nonblocking socket I/O.
- Linux event loops.
- HTTP protocol processing.
- Memory safety.
- Concurrency and resource management.
- Performance profiling.

PulseHTTP adds an important systems-programming dimension to the portfolio without duplicating the main purpose of the other projects.

---

22. Final Deliverables

The finished repository should contain:

- Complete C++20 source code.
- A reusable server interface.
- An epoll-based reactor.
- An incremental HTTP parser.
- A bounded worker pool.
- Safe connection lifecycle management.
- Static file serving.
- Configurable resource limits.
- Unit and integration tests.
- Fuzz and sanitizer tests.
- Prometheus metrics.
- Docker deployment.
- Benchmark scripts.
- Raw performance measurements.
- Comparative performance charts.
- A technical case study.
- Clear reproduction instructions.

Final success criterion: A reviewer can build and run the server, inspect its concurrency and memory-safety design, reproduce the benchmarks on compatible Linux hardware, and verify that every performance claim is supported by measured results.